

Gestion des entrées / sorties

Plan

Les flux : présentation et dénomination

Les flux de caractères

Les flux d'octets

Les flux d'objets : la sérialisation

La classe Scanner

Présentation des flux

Un programme échange des informations : réception d'une source ou envoi vers une destination (fichiers, sockets réseau, autres programmes...) des informations de nature potentiellement hétérogène (texte, image, son...).

On distingue :

- ▶ la gestion et les traitements du système
 - ▶ création/suppression de fichier
 - ▶ exploration d'arborescence...
 - la classe `File`
- ▶ la gestion et les traitements sur les flux d'informations
 - ▶ lecture à partir d'un fichier
 - ▶ écriture sur une *socket*
 - les classes `Reader` ou `InputStream`

Présentation des flux

Les flux (*streams* en anglais) permettent d'encapsuler ces processus d'envoi et de réception de données. Les flux traitent toujours les données de façon séquentielle.

Il existe 3 flux prédéfinis :

- ▶ l'entrée standard `System.in`
- ▶ la sortie standard `System.out`
- ▶ la sortie d'erreurs `System.err`

En Java, les flux peuvent être divisés en plusieurs catégories selon deux critères :

- ▶ l'orientation du flux
 - ▶ les flux d'entrée (*input stream*)
 - ▶ les flux de sortie (*output stream*)
- ▶ la nature des informations traitées
 - ▶ les flux de traitement de caractères
 - ▶ les flux de traitement d'octets

Présentation des flux

Java définit des flux pour lire ou écrire des données, mais aussi des classes qui, associée à un flux, permettent de faire des **traitements** sur les données du flux : des **filtres**.

Par exemple, il existe des filtres qui permettent de mettre les données traitées dans un tampon (buffer) pour les traiter par lots.

Toutes ces classes sont regroupées dans le package `java.io`. On trouve un total de 29 classes :

- ▶ 16 classes de flux entrants / 13 classes de flux sortants
- ▶ 14 classes sur les caractères / 15 classes sur les octets

Une difficulté : parmi toutes ces classes, choisir celle qui convient le mieux en fonction des besoins. Pour faciliter ce choix, il faut comprendre la dénomination des classes.

Règle de dénomination

Les suffixes

Le nom des classes se compose d'un *préfixe* et d'un *suffixe*. Il y a quatre suffixes possibles en fonction du type de flux (flux d'octets ou de caractères) et du sens du flux (entrée ou sortie).

	Flux d'octets	Flux de caractères
Flux d'entrée	<code>InputStream</code>	<code>Reader</code>
Flux de sortie	<code>OutputStream</code>	<code>Writer</code>

À partir de ces quatre classes abstraites, il existe donc quatre hiérarchies de classes qui représentent des types de flux particuliers, selon le sens et le type du flux.

`InputStream`

`Reader`

`OutputStream`

`Writer`

Règle de dénomination

Les préfixes

Pour les préfixes, il faut distinguer les flux des filtres (traitement sur des flux). Pour les flux, le préfixe contient la source ou la destination (selon le sens du flux) :

- ▶ `ByteArray` : tableau d'octets en mémoire
- ▶ `CharArray` : tableau de caractères en mémoire
- ▶ `File` : fichier
- ▶ `Object` : objet
- ▶ `Pipe` : pipeline entre deux threads
- ▶ `String` : chaîne de caractères

Exemples de combinaisons possibles :

- ▶ `StringReader` : définit un flux de caractères en lecture à partir d'une chaîne ;
- ▶ `CharArrayOutputStream` : buffer de caractères de destination d'un flux de sortie, données que l'on peut récupérer via une méthode `toCharArray()` ou `toString()`...

Règle de dénomination

Les préfixes

Pour les filtres, le préfixe contient le type de traitement qu'il effectue. Les filtres **n'existent pas obligatoirement** pour des flux en entrée et en sortie, ou des caractères et des octets.

- ▶ `Buffered` : mise des données du flux dans un tampon (*E/S*)
- ▶ `Sequence` : fusion de plusieurs flux (*E-byte*)
- ▶ `Data` : traitement des octets sous forme de type de données (*E/S-byte*)
- ▶ `LineNumber` : numérotation des lignes du flux (*E-char*)
- ▶ `PushBack` : remise des données lues dans le flux (*E*)
- ▶ `Print` : impressions formatées (*S*)
- ▶ `Object` : sérialisation (*E/S-byte*)
- ▶ `InputStream/OutputStream` : conversion des octets en caractères (*E/S-byte*)

Exemples de combinaisons possibles :

- ▶ `BufferedInputStream` : définit un tampon pour le traitement par lots ;
- ▶ `ObjectOutputStream` : écriture d'objets sous forme de flux d'octets...

Package java.io

Pour résumer, 29 classes...

Les flux de caractères en lecture :

- ▶ BufferedReader
- ▶ CharArrayReader
- ▶ FileReader
- ▶ InputStreamReader
- ▶ LineNumberReader
- ▶ PipedReader
- ▶ PushbackReader
- ▶ StringReader

Les flux de caractères en sortie :

- ▶ BufferedWriter
- ▶ CharArrayWriter
- ▶ FileWriter
- ▶ OutputStreamWriter
- ▶ PipedWriter
- ▶ StringWriter

Les flux d'octets en lecture :

- ▶ BufferedInputStream
- ▶ ByteArrayInputStream
- ▶ DataInputStream
- ▶ FileInputStream
- ▶ ObjectInputStream
- ▶ PipedInputStream
- ▶ PushbackInputStream
- ▶ SequenceInputStream

Les flux d'octets en sortie :

- ▶ BufferedOutputStream
- ▶ ByteArrayOutputStream
- ▶ DataOutputStream
- ▶ FileOutputStream
- ▶ ObjectOutputStream
- ▶ PipedOutputStream
- ▶ PrintStream

Plan

Les flux : présentation et dénomination

Les flux de caractères

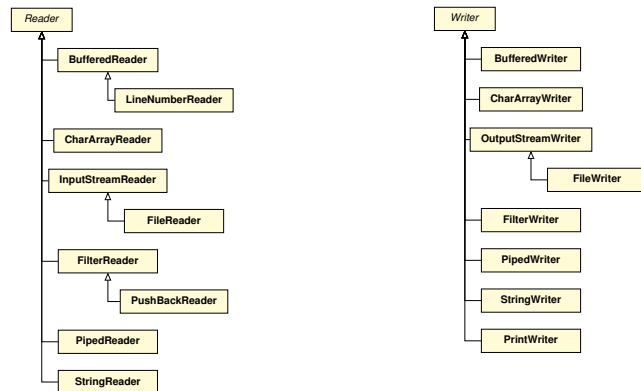
Les flux d'octets

Les flux d'objets : la sérialisation

La classe Scanner

Les flux de caractères

Ces flux transportent des données sous forme de caractères (selon le format *Unicode* sur 2 octets).



Lecture de caractères

La classe Reader

C'est une classe abstraite, mère de toutes les classes qui gèrent des flux de caractères en lecture, notamment `CharArrayReader`, `FileReader` et `StringReader` pour la lecture à partir de tableaux de caractères, de fichiers ou de chaînes de caractères.

- ▶ `boolean ready()` : indique si le flux est prêt à être lu
- ▶ `void close()` : ferme le flux et libère les ressources qui lui étaient associées
- ▶ `int read()` : renvoie le caractère lu ou -1 si la fin du flux est atteinte
- ▶ `int read(char[])` : lit plusieurs caractères et les met dans un tableau de caractères
- ▶ `long skip(long n)` : saute autant de caractères dans le flux que spécifié et renvoie le nombre de caractères sautés

Il est également possible de marquer des positions spécifiques dans un flux : `markSupported()`, `mark()`, `reset()`...

Lecture de caractères

Exemple : la classe `FileReader`

Intéressons-nous à la lecture de caractères à partir d'un fichier.

```
try (Reader fr = new FileReader(myPath+sourceFile)) {
    StringBuilder sb = new StringBuilder();
    int c = fr.read();
    while (c != -1) {
        sb.append((char) c);
        c = fr.read();
    }
    System.out.println(sb.toString());
} catch (FileNotFoundException e) {
    System.out.println("File not found: "); e.printStackTrace();
} catch (IOException e) {
    System.out.println("Reading error: " + e.getMessage());
    e.printStackTrace();
}
```

On peut remarquer l'inconvénient majeur : la méthode `read()` est invoquée autant de fois qu'il y a de caractères dans le fichier...

Les flux "tamponnés"

Pour éviter cet inconvénient, il faut mettre en tampon (zone mémoire à accès rapide) les caractères lus afin de les traiter par lots (une fois que cette zone arrive à saturation). C'est le rôle de la classe `BufferedReader`.

Les constructeurs `BufferedReader(Reader in)` ou `BufferedReader(Reader in, int sz)` permettent la création d'un flux de lecture avec un tampon ("*buffer*") de taille éventuellement paramétrable.

Toute instance doit se combiner avec un autre `Reader`. En plus de sa fonctionnalité de mise en buffer, elle expose une méthode très utile `readLine()` qui retourne une chaîne de caractères contenant la ligne lue.

Cette classe est étendue par `LineNumberReader`, qui permet en plus de récupérer le numéro de ligne.

Lecture de caractères

Exemple : la classe `BufferedReader`

Voici un exemple complet de lecture "*tamponnée*" de fichiers de caractères.

```
try (BufferedReader br = new BufferedReader(
    new FileReader(myPath+sourceFile))) {
    StringBuilder sb = new StringBuilder();
    String line = br.readLine();
    while (line != null) {
        sb.append(line);
        line = br.readLine();
    }
    System.out.println(sb.toString());
} catch (FileNotFoundException e) {
    System.out.println("File not found: "); e.printStackTrace();
} catch (IOException e) {
    System.out.println("Reading error: " + e.getMessage());
    e.printStackTrace();
}
```

Seules la création de l'instance (un `FileReader` en paramètre du constructeur du `BufferedReader`) et l'invocation de la méthode `readLine()` (au lieu de `read()`) diffèrent.

Écriture de caractères

La classe `Writer`

La classe abstraite `Writer` expose les méthodes suivantes :

- ▶ `void write(char[] buffer)`, `void write(String s)`, et `void write(int c)` et leurs surcharges permettent l'écriture de caractères respectivement à partir d'un tableau, d'une chaîne ou d'un entier.
- ▶ `void append(char c)` et ses surcharges fonctionnent de manière analogue, mais renvoient une instance de `Writer`.
- ▶ `void flush()` permet de vider les tampons d'écriture vers le médium de sortie.
- ▶ `void close()` ferme ce flux.

Cette classe est dérivée en `FileWriter`, `StringWriter` et `CharArrayWriter` qui correspondent à différents supports de sortie.

Écriture de caractères

Exemple : la classe `FileWriter`

Intéressons-nous à l'écriture de caractères dans un fichier.

```
double[] data = {15.9, 21.2, 18.4, 25.4, 31.1};
try (Writer fw = new FileWriter(myPath+destinationFile)) {
    for (double d : data) {
        fw.write (String.valueOf(d));
        fw.write ("\r\n");
    }
} catch (IOException e) {
    System.out.println("Writing error: " + e.getMessage());
    e.printStackTrace();
}
```

L'inconvénient majeur est le même que pour la lecture : la méthode `write()` est invoquée autant de fois qu'il y a de caractères à écrire...

Écriture de caractères

Exemple : la classe `BufferedWriter`

Voici un exemple complet d'écriture *"tamponnée"* dans un fichier.

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter(myPath+destinationFile))) {
    double number = 42.0;
    bw.write("Hello everybody");
    bw.newLine();
    bw.write("We are the " + LocalDate.now());
    bw.write(", the ultimate number is " + number);
} catch (IOException e) {
    System.out.println("Writing error: " + e.getMessage());
    e.printStackTrace();
}
```

On obtient alors un fichier contenant les lignes suivantes :

Hello everybody

We are the 2020-05-03, the ultimate number is 42.0

Écriture de caractères

La classe `PrintWriter`

La classe `PrintWriter` étend la classe `Writer` et à l'instar de `BufferedWriter`, fonctionne sur un flux passé en paramètre.

Elle expose des méthodes bien connues en C : `print()`, `println()` et `printf()` permettant de formater des affichages.

Notons que pour l'écriture dans des fichiers, la classe `PrintWriter` expose également un constructeur qui prend une instance de `File`...

```
try (PrintWriter pw = new PrintWriter(new File(myPath+destinationFile))) {
    double number = 42.0;
    pw.println("Hello everybody");
    pw.print("We are the " + LocalDate.now());
    pw.println(", the ultimate number is " + number);
} catch (IOException e) {
    System.out.println("Writing error: " + e.getMessage());
    e.printStackTrace();
}
```

Avant d'aller plus loin...

Un point de style...

Différents "styles" d'écriture sont possibles. Pour les flux, on peut opter pour des déclarations indépendantes :

```
File f = new File(myPath+destinationFile);
FileWriter fw = new FileWriter(f);
BufferedWriter bw1 = new BufferedWriter(fw);
```

On peut aussi compacter cette écriture :

```
BufferedWriter bw2 = new BufferedWriter(
    new FileWriter(new File(myPath+destinationFile)));
```

Les variables intermédiaires ne sont pas manipulables dans le second cas. Il faut adapter votre style d'écriture à vos besoins :

- ▶ manipulation d'un flux sur plusieurs fichiers
- ▶ fusion de flux dans une même sortie
- ▶ ...

Plan

Les flux : présentation et dénomination

Les flux de caractères

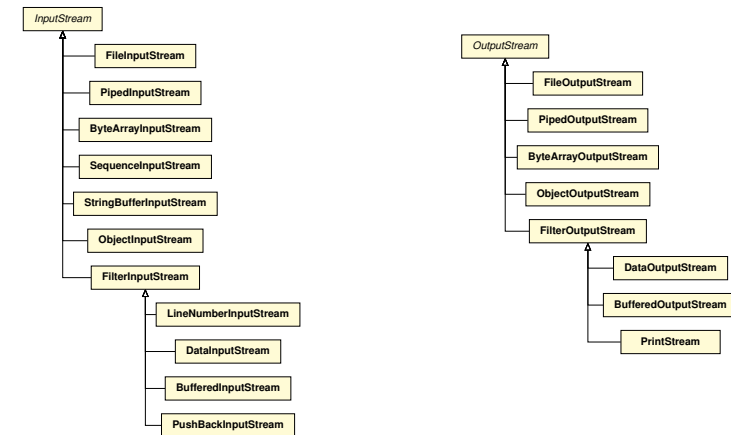
Les flux d'octets

Les flux d'objets : la sérialisation

La classe Scanner

Les flux d'octets

Ces flux transportent des données sous forme d'octets. Les flux de ce type sont capables de traiter toutes les données.



Manipulation de flux d'octets

Les classes `InputStream` et `OutputStream`

Ces classes abstraites suivent le même schéma de fonctionnement que celui des classes `Reader` et `Writer` dédiées aux flux de caractères.

La classe `InputStream` expose :

- ▶ `read()` retourne l'octet lu et ses surcharges permettent de lire un ou des octet(s) et retournent toutes le nombre d'octets effectivement lus. Toutes retournent -1 en fin de flux.
- ▶ `skip(long n)` : saute le nombre d'octets passé en paramètre dans le flux de lecture.
- ▶ `reset()` : réinitialise la lecture de ce flux.
- ▶ `available()` : retourne le nombre d'octets disponibles dans un `int`.

La classe `OutputStream` expose :

- ▶ `write(int b)` et ses surcharges : permettent d'écrire des octets directement sur le flux.
- ▶ `close()` et `flush()` qui ont la même sémantique que celles de la classe `Writer`.

Manipulation de flux d'octets

Un exemple : `FileOutputStream`

L'avantage du traitement sur des flux d'octets est leur polyvalence : tout type de données peut être utilisé. Ils sont à privilégier dès qu'une exploration manuelle n'est pas nécessaire (*human-readable*).

```
byte[] byteArray = {0,1,2};
try(OutputStream fos = new FileOutputStream(new File(myPath+destinationFile))) {
    fos.write(byteArray);
} catch(IOException e) {
    System.out.println("Error " + e.getMessage());
    e.printStackTrace();
}
```

Pour améliorer les performances des flux sur un fichier, la mise en tampon des données lues ou écrites permet de traiter un ensemble d'octets plutôt que de traiter les données octet par octet. Le nombre d'opérations est ainsi réduit.

Un type particulier de flux binaire

Les classes `ObjectOutputStream` et `ObjectInputStream`

La classe `ObjectOutputStream` supporte l'écriture directe d'objets sur des flux alors que l'objet `ObjectInputStream` permet la lecture de flux contenant des objets.

Au travers de ce mécanisme, Java fournit une façon facile, transparente et standard de réaliser un mécanisme de persistance, évitant de créer un format particulier pour sauvegarder et relire un objet.

Un tel mécanisme peut être utilisé pour :

- ▶ stocker des objets momentanément inutilisés
- ▶ sauvegarder une configuration
- ▶ échanger des objets entre plusieurs JVM
- ▶ rendre persistants des objets (dans un fichier ou une base de données par exemple)
- ▶ créer une copie intégrale d'un objet (*deep copy*)
- ▶ ...

Plan

Les flux : présentation et dénomination

Les flux de caractères

Les flux d'octets

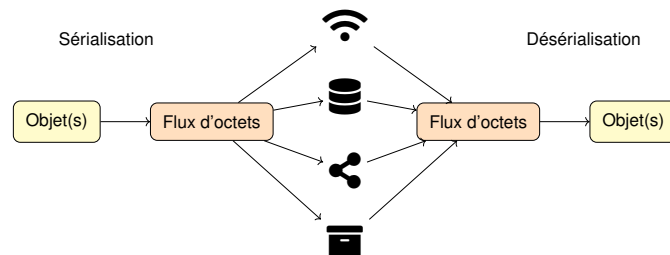
Les flux d'objets : la sérialisation

La classe `Scanner`

La sérialisation

Le principe

Ce mécanisme particulier s'appelle **serialization**, et a un fonctionnement particulier.



La sérialisation peut s'appliquer facilement à quasiment tous les objets. C'est en revanche impossible pour les classes qui sont liées à des éléments du système comme `Thread`, `OutputStream` et ses sous-classes, `Socket`, `Image`...

La sérialisation

En pratique

Il existe plusieurs formats de sérialisation appartenant à deux grandes familles :

- ▶ formats binaires : c'est le format par défaut
- ▶ formats textes : ils se basent sur une structuration standard (XML, JSON...). Ils sont plus portables, plus facilement modifiables, mais consomment beaucoup plus de ressources pour être traités.

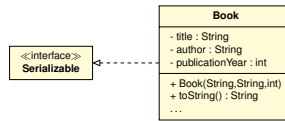
Pour pouvoir être sérialisée, une classe doit implémenter l'interface `java.io.Serializable`. Cette interface ne contient aucune méthode à implémenter, elle agit simplement comme un marqueur.

Les méthodes nécessaires à l'écriture et la lecture sont fournies par les classes `ObjectOutputStream` et `ObjectInputStream`, notamment :

- ▶ `void writeObject(Object obj)` pour la sérialisation
- ▶ `Object readObject()` pour la désérialisation

La sérialisation

Un exemple



Pour sérialiser un tel objet :

```
try (ObjectOutputStream oos
    = new ObjectOutputStream(
        new FileOutputStream(new File(myPath+myFile)))) {
    oos.writeObject(new Book("Flatland", "E.Abbott Abbott", 1884));
} catch (Exception e) {e.printStackTrace();}
```

Pour désérialiser cet objet :

```
try (ObjectInputStream ois
    = new ObjectInputStream(
        new FileInputStream(new File(myPath+myFile)))) {
    Book tmp = (Book)ois.readObject();
    System.out.println(tmp);
} catch (Exception e) {e.printStackTrace();}
```

La sérialisation

Les caractéristiques et les limites

Ce mécanisme est celui proposé par défaut, mais peut être surchargé. Il dépend de la structure de l'objet : l'ajout ou la suppression d'un champ peut modifier le comportement du mécanisme de sérialisation par défaut.

Par exemple, par défaut si une classe `Serializable` ne déclare pas explicitement un champ `serialVersionUID` alors le compilateur va l'ajouter et calculer une valeur qui prend en compte les différents éléments qui composent la classe.

Si la classe est modifiée, le champ `serialVersionUID` sera recalculé avec une version différente. Les données des sérialisations précédentes lèveront une `InvalidClassException`.

La sérialisation

Les caractéristiques et les limites

On peut préciser qu'un champ ne doit pas être pris en compte lors d'une sérialisation en ajoutant le mot-clé `transient` lors de sa déclaration.

- ▶ Le contenu des attributs est visible dans le flux dans lequel est sérialisé l'objet. Même si le champ est `private`, toute personne ayant accès au flux peut consulter les valeurs. Ce qui peut poser des problèmes de sécurité surtout si les données sont sensibles.
- ▶ Lors d'un processus de désérialisation, les attributs `transient` sont mis à `null`.
- ▶ Tout champ non sérialisable (comme un `Thread`) doit également être marqué `transient` au risque de lever une exception `NotSerializableException`

La sérialisation

Les caractéristiques et les limites

Plusieurs points particuliers doivent être pris en compte lors de la mise en œuvre de la sérialisation/désérialisation.

- ▶ Les membres `static` d'un objet ne sont jamais sérialisés.
 - ▶ Par exemple, trois instances d'une classe sont sérialisées à des moments où la valeur du membre `static` est différente... Lors de la désérialisation, quelle serait la bonne valeur pour ce membre ?
- ▶ Si une classe mère implémente l'interface `Serializable` alors toutes les classes filles en héritent.
 - ▶ Il est donc inutile d'implémenter explicitement cette interface pour une classe fille. Il n'est pas facile de savoir si une classe est sérialisable en regardant son code.
- ▶ Si une classe mère n'est pas sérialisable, l'état des attributs hérités sont ignorés, la valeur est celle par défaut selon le type.

La sérialisation personnalisée

Il est possible de surcharger les méthodes `writeObject()` et `readObject()` dans la classe à sérialiser. Les signatures doivent être **impérativement** :

- ▶ `private void readObject(ObjectInputStream ois) throws IOException, ClassNotFoundException`
- ▶ `private void writeObject(ObjectOutputStream oos) throws IOException`

Lors de la sérialisation/désérialisation d'un objet de la classe, ces deux méthodes seront invoquées en remplacement du mécanisme standard.

```
private void readObject(ObjectInputStream ois)
    throws IOException, ClassNotFoundException {
    this.title = (String)ois.readObject();
    this.author = (String)ois.readObject();
}
private void writeObject(ObjectOutputStream oos) throws IOException {
    oos.writeObject(this.title);
    oos.writeObject(this.author);
}
```

La sérialisation personnalisée

- ▶ Le mécanisme de personnalisation peut permettre de sérialiser des champs privés d'une classe mère qui n'est pas sérialisable.
- ▶ Il est toujours possible d'utiliser le mécanisme de sérialisation par défaut en invoquant les méthodes
 - ▶ `defaultReadObject()` de la classe `ObjectInputStream`
 - ▶ `defaultWriteObject()` de la classe `ObjectOutputStream`
- ▶ L'interface `Externalizable` dérive de `Serializable` et permet d'avoir un contrôle très fin des opérations :
 - ▶ par défaut, aucun attribut n'est pris en compte (`transient` est inutile)
 - ▶ il est possible de modifier les valeurs de certaines données de l'état de l'objet

Plan

Les flux : présentation et dénomination

Les flux de caractères

Les flux d'octets

Les flux d'objets : la sérialisation

La classe `Scanner`

La classe `Scanner`

Dans le paquetage `java.util`, la classe `Scanner` facilite l'analyse et le traitement des types primitifs dans un flux à la manière d'un itérateur.

On peut lire une valeur sur l'entrée standard `System.in`, lire un fichier ou même décomposer une chaîne de caractères. La classe `Scanner` bénéficie de 10 constructeurs...

Un exemple très simple à partir de l'entrée standard :

```
Scanner sc = new Scanner(System.in);
int i = sc.nextInt();
System.out.println(i);
```

Un exemple de lecture à partir d'une chaîne :

```
String input = "1 2 red blue";
Scanner sc = new Scanner(input);
System.out.println(sc.nextInt());
System.out.println(sc.nextInt());
System.out.println(sc.next());
System.out.println(sc.next());
sc.close();
```

Cette apparente simplicité s'accompagne de contraintes :

- l'utilisation d'un `Scanner` n'est possible que si l'on connaît la structure du flux

Le code suivant mène à l'exception `InputMismatchException` :

```
String input = "1 2 red blue 12";
Scanner sc = new Scanner(input);
System.out.println(sc.nextInt());
System.out.println(sc.nextInt());
System.out.println(sc.nextInt());
sc.close();
```

En revanche, c'est très pratique pour la lecture de fichier de données dont la structure est connue à l'avance, comme les fichiers CSV par exemple.